Rapport du Projet

MTH8211

Oussama Mouhtal, Yasmine Amami

Toy example for Data Assimilation

The aim of data assimilation is to reduce the uncertainties in models with the help of observations. In this notebook the Kalman Filter and the 4D-Var methods are compared. First with a linear model and then with the classical Lorenz 1969 model.

```
using DataAssim
using LinearAlgebra
using Plots
using Random
using Test
```

```
Precompiling DataAssim...
   2285.6 ms  DataAssim
  1 dependency successfully precompiled in 3 seconds. 214 already precompiled.
```

State vector

The state vector $\vec{x}_i$ containing all *prognostic* variables at time t_i (time of the i-th time step).

Model

A model   need to define the following operations :

- a foreward step : `(t,x, )`
- tangent linear model : `tgl( ,t,x,dx)`
- adjoint model : `adj( ,t,x,dx2)`

  is random noise for a stochastic model or zero for a detministic model.

```
function check( ::AbstractModel,n,t = 0,  = 1e-4)
    dx = randn(n)
    x = randn(n)
    dx2 = randn(n)
    @test ( (t,x +  .*dx) -  (t,x -  .*dx)) / (2* )    tgl( ,t,x,dx) atol=10* ^2
    @test dx2   tgl( ,t,x,dx)   adj( ,t,x,dx2)   dx    atol=1e-7

    dX = randn(n,3)
    MdX = tgl( ,t,x,dX)
    @test tgl( ,t,x,dX[:,1])   MdX[:,1]
end
```

```
check (generic function with 3 methods)
```

Simple model can be defined using matrix `ModelMatrix(M)`

$$\vec{x}_{i+1} = \mathbf{M}\,\vec{x}_i$$

```
 = ModelMatrix(2*I)

x = randn(90)
@test  (0,x)   2*x
@test tgl( ,0,0,x)   2*x
@test adj( ,0,0,x)   2*x
check( ,4)
```

```
Test Passed
```

Or using a operator M, potentially non-linear :

$$\vec{x}_{i+1} = M(t_i, \vec{x}_i)$$

In this case, the tagent linear and adjoint model have to be provided too (or derived from automatic differential tools)

```
 = ModelFun((t,x, ) -> 2*x,(t,x,dx) -> 2*dx,(t,x,dx) -> 2*dx)

x = randn(10)
@test  (0,x)   2*x
@test tgl( ,0,0,x)   2*x
@test adj( ,0,0,x)   2*x
check( ,10)
```

```
Test Passed
```

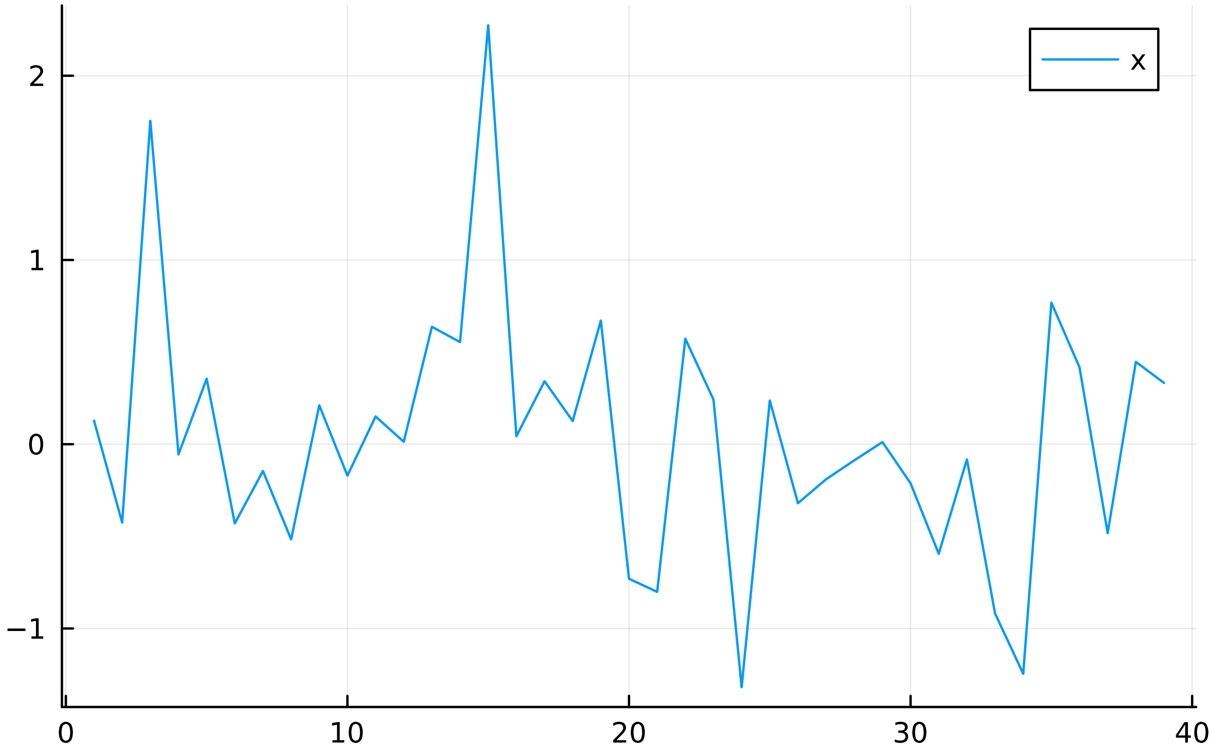
Shallow water 1D model

```
dt = 0.5
g = 9.81
h = 100.0
L = 10000.0
n = 20

 = LinShallowWater1DModel(dt, g, h, L, n)
check( ,2*n-1)
```

```
Test Passed
```

```
 0 = randn(n - 1)
u0 = randn(n)
x = pack( ,  0, u0)
p = Plots.plot(x, label="x")
savefig("Mx.pdf")
```

```
"/Users/mouhtal/DataAssim.jl/examples/Mx.pdf"
```

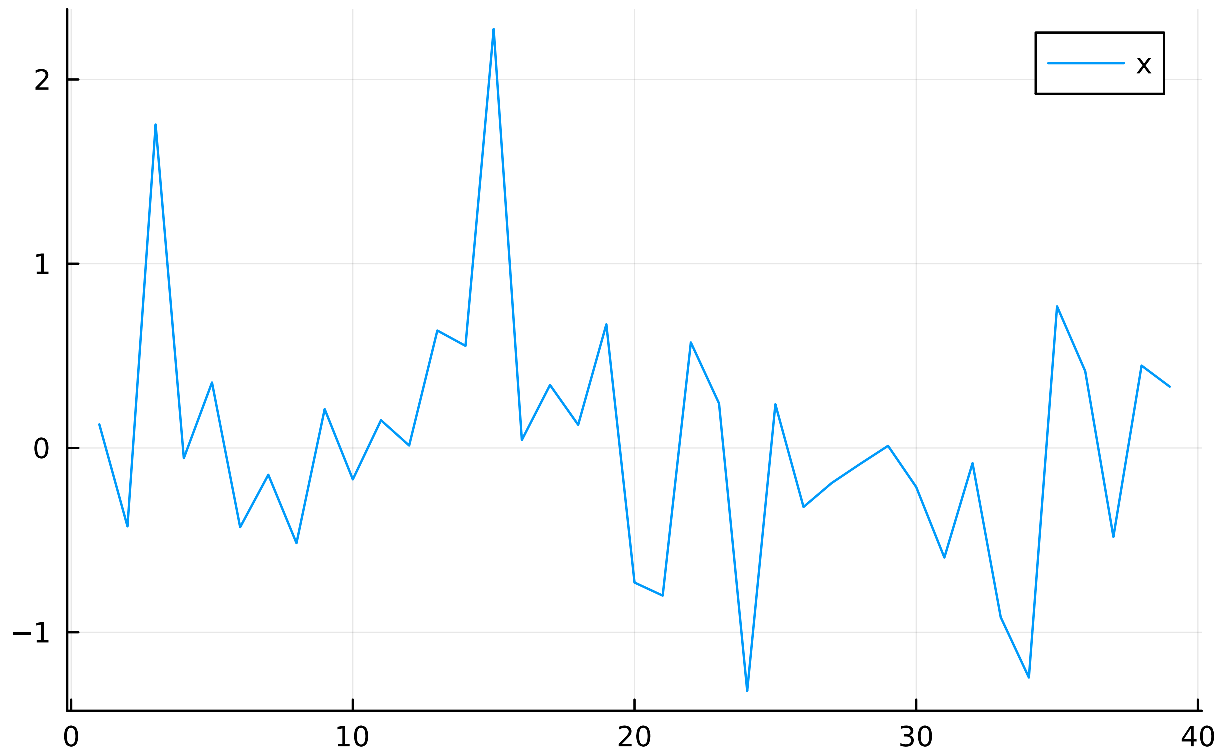


;

```
#dt = 1.0
#g = 9.81
#h = 100
#n = 101
#L = 10000.0
m = 10

#  = LinShallowWater1DModel(dt, g, h, L, n)

nmax = 4;
no = 1:2:nmax;
 0 = randn(n - 1)
u0 = randn(n)
xit = pack( ,  0, u0);
H = Matrix(I, m, 2*n-1);
  = ModelMatrix(H)
Pi = Matrix(0.5*I,2*n-1,2*n-1)
Q = zeros(2*n-1,2*n-1)

R = [Matrix(0.5*I, m, m) for n in no]
xt, xfree, xa, yt, yo, diag_ , Aerrors = TwinExperiment( , xit, Pi, Q, R,  , nmax, no, "4DVar")
savefig("Aerrors.pdf")
```

```
39×39 Matrix{Float64}:
 6.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  …  0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  6.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  6.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  6.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  6.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
```

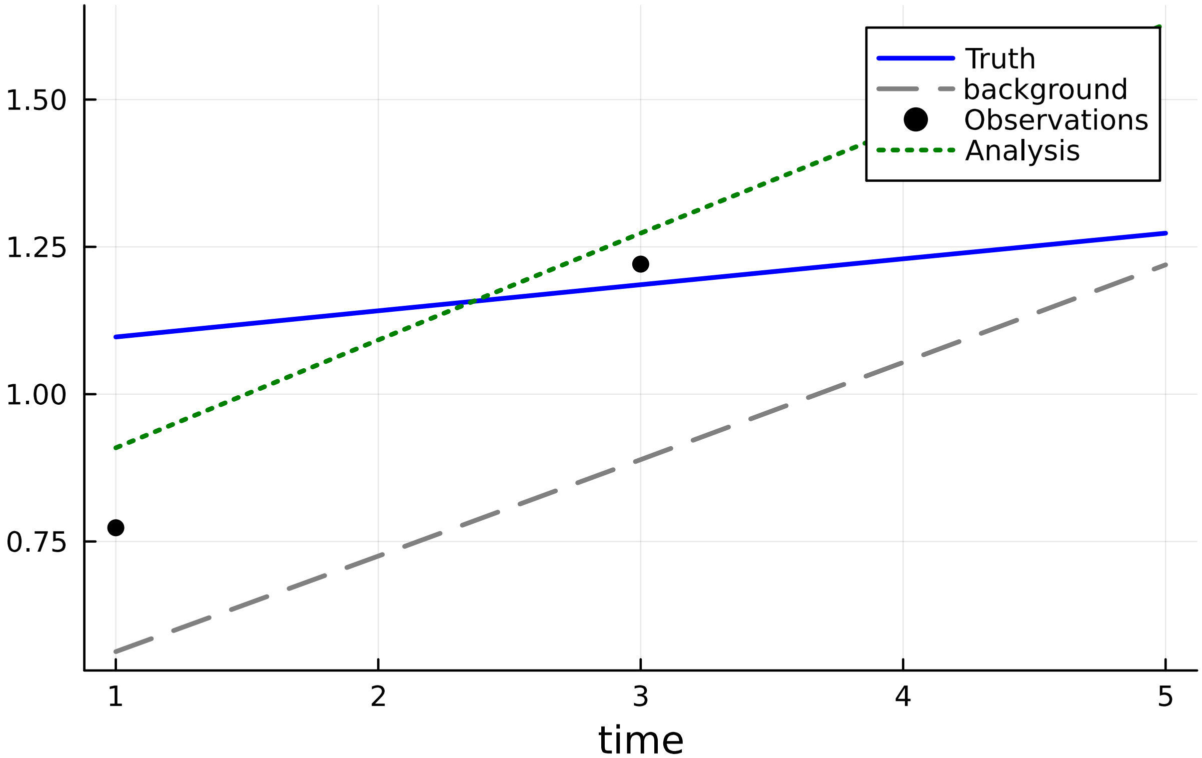
```
 0.0  0.0  0.0  0.0  0.0  6.0  0.0  0.0  …  0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  6.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  6.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0

 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  …  0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     2.0  0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  2.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  2.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  …  0.0  0.0  0.0  2.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  2.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  2.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0     0.0  0.0  0.0  0.0  0.0  0.0  2.0
39-element Vector{Float64}:
 1.9999999999999962
 1.9999999999999987
 1.9999999999999987
 1.9999999999999991
 1.9999999999999993
 1.9999999999999996
 1.9999999999999998
 1.9999999999999998
 1.9999999999999998
 1.9999999999999998
 ⋮
 6.011461494929598
 6.024919055009145
 6.042207036061259
 6.0619248686766705
 6.082475131323333
 6.102192963938732
 6.119480944990848
 6.132938505070408
 6.141475392694973
"/Users/mouhtal/DataAssim.jl/examples/Aerrors.pdf"
```



;

```
time = 1:nmax+1

p = Plots.plot(
    time, xt[2, :],
    label = "Truth", lw = 2, color = :blue,
    xlabel = "time",
    legend = :topright,
    grid = :on,
)

Plots.plot!(
    p, time, xfree[2, :],
    label = "background", lw = 2, linestyle = :dash, color = :gray
)

Plots.plot!(
    p, time[no], getindex.(yo, 2),
    label = "Observations", seriestype = :scatter, marker = :circle, color = :black
)

Plots.plot!(
    p, time, xa[2, :],
    label = "Analysis", lw = 2, linestyle = :dot, color = :green
)
savefig("Assimilation.pdf")
```

```
"/Users/mouhtal/DataAssim.jl/examples/Assimilation.pdf"
```


Truth
background
Observations
Analysis
1.50
1.25
1.00
0.75
1
2
3
4
5
time

;